

BÁO CÁO KẾT QUẢ THỬ NGHIỆM

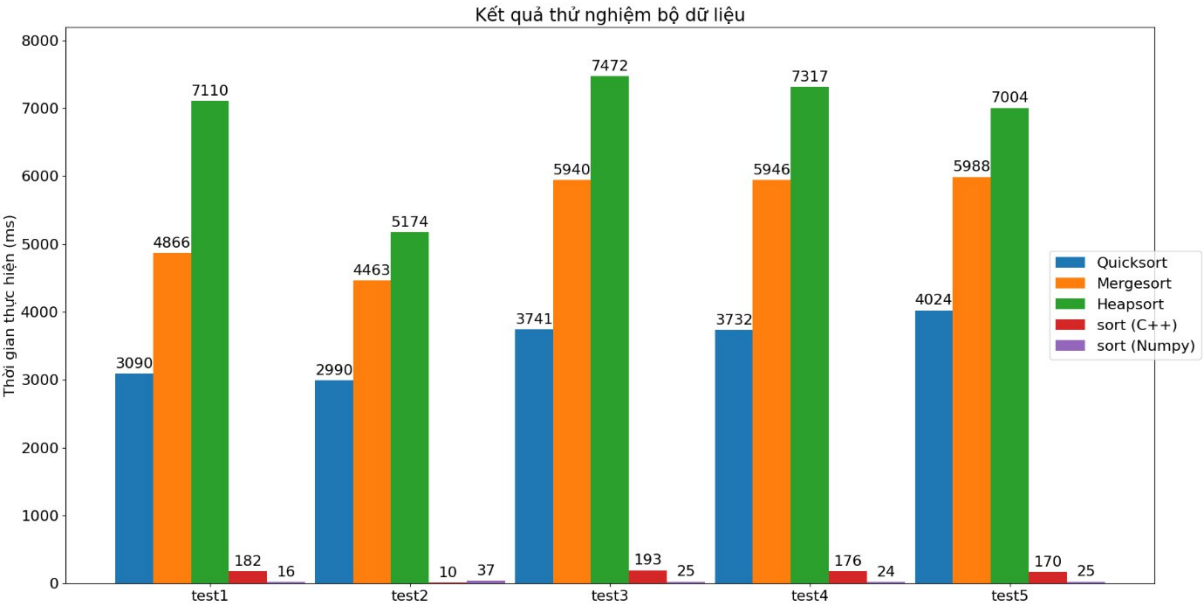
Sinh viên thực hiện: Trần Bảo Đăng - 25520244

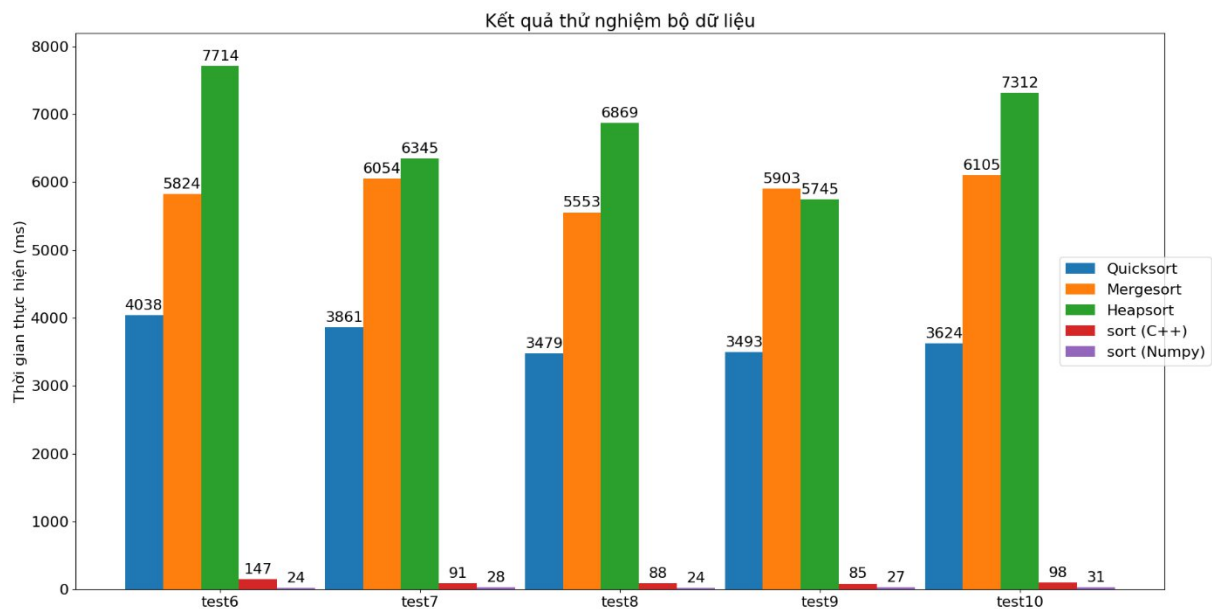
Nội dung báo cáo:

I. Kết quả thử nghiệm
1. Bảng thời gian thực hiện

Dữ liệu	Thời gian thực hiện (ms)				
	Quicksort	Heapsort	Mergesort	sort (C++)	sort (numpy)
1 (số thực, tăng dần)	3090	4866	7110	182	16
2 (số nguyên, giảm dần)	2990	4463	5174	10	37
3 (số thực)	3741	5940	7472	193	25
4 (số thực)	3732	5946	7317	176	24
5 (số thực)	4024	5988	7004	170	25
6 (số thực)	4038	5824	7714	147	24
7 (số nguyên)	3861	6054	6345	91	28
8 (số nguyên)	3479	5553	6869	88	24
9 (số nguyên)	3493	5903	5745	85	27
10 (số nguyên)	3624	6105	7312	98	31
Trung bình	3607,2	5664,2	6806,2	124	26,1

2. Biểu đồ (cột) thời gian thực hiện





II. Kết luận:

Kết quả cho thấy sự khác biệt rõ rệt về hiệu năng giữa các thuật toán không chỉ phụ thuộc vào độ phức tạp lý thuyết mà còn chịu ảnh hưởng rất lớn bởi ngôn ngữ lập trình và mức độ tối ưu của cài đặt. Trong ba thuật toán được tự cài đặt bằng Python, Quicksort cho hiệu năng tốt nhất, tiếp theo là Mergesort và Heapsort; tuy nhiên, tất cả đều chậm hơn đáng kể so với các hàm sắp xếp được cung cấp sẵn trong thư viện chuẩn. Đặc biệt, hàm sort của C++ và numpy.sort cho thời gian chạy vượt trội, trong đó numpy.sort đạt hiệu năng cao nhất nhờ được hiện thực bằng ngôn ngữ biên dịch và tối ưu cho xử lý mảng số lớn. Ngoài ra, kết quả thực nghiệm cũng cho thấy thứ tự ban đầu của dữ liệu (ngẫu nhiên, đã sắp xếp tăng hoặc giảm) và kiểu dữ liệu (số nguyên hay số thực) không gây ảnh hưởng đáng kể đến hiệu năng của các hàm thư viện tối ưu. Từ đó có thể kết luận rằng, đối với các bài toán sắp xếp dữ liệu lớn trong thực tế, việc sử dụng các thư viện chuẩn hoặc thư viện tối ưu sẵn có là lựa chọn hiệu quả và phù hợp hơn so với việc tự cài đặt thuật toán bằng Python thuần, đặc biệt khi mục tiêu chính là hiệu năng.

III. Thông tin chi tiết

<https://github.com/Lilinta/IT003-Sorting-Algorithms-Benchmark>